

Round 2 Technical Report: Semantic Comprehension Layer Rebuild

Team: Kislay Jha, Vedansh Shah

1. Problem Definition

The Social Engine's semantic comprehension layer experienced a critical failure, rendering it unable to interpret the tone and intent of human language. The objective of this phase is to rebuild this NLP module by training a machine learning model capable of accurately classifying social media text into three distinct sentiment categories: Positive, Negative, and Neutral.

2. Preprocessing Pipeline

Social media text is inherently noisy. We designed a robust preprocessing pipeline using Python's regular expressions (`re`) to standardize the textual data before vectorization:

- **Normalization:** Converted all text to lowercase to ensure uniformity.
- **Noise Removal:** Stripped URLs, user mentions (@), hashtags (#), and special punctuation to prevent the model from learning irrelevant, non-semantic patterns.
- **Feature Extraction:** Utilized Scikit-learn's `TfidfVectorizer` to convert text into numerical weights. We restricted the vocabulary to the top 5,000 features, removed standard English stopwords, and utilized bi-grams (`ngram_range=(1, 2)`) to capture contextual phrases rather than just isolated words.

3. Model Selection

We selected **Logistic Regression** as our primary classification engine. For text classification problems involving high-dimensional sparse matrices (like TF-IDF outputs), Logistic Regression serves as an excellent, computationally efficient baseline. It is highly interpretable, scales well with large datasets, and is less prone to overfitting compared to complex tree-based models when dealing with vast textual vocabularies.

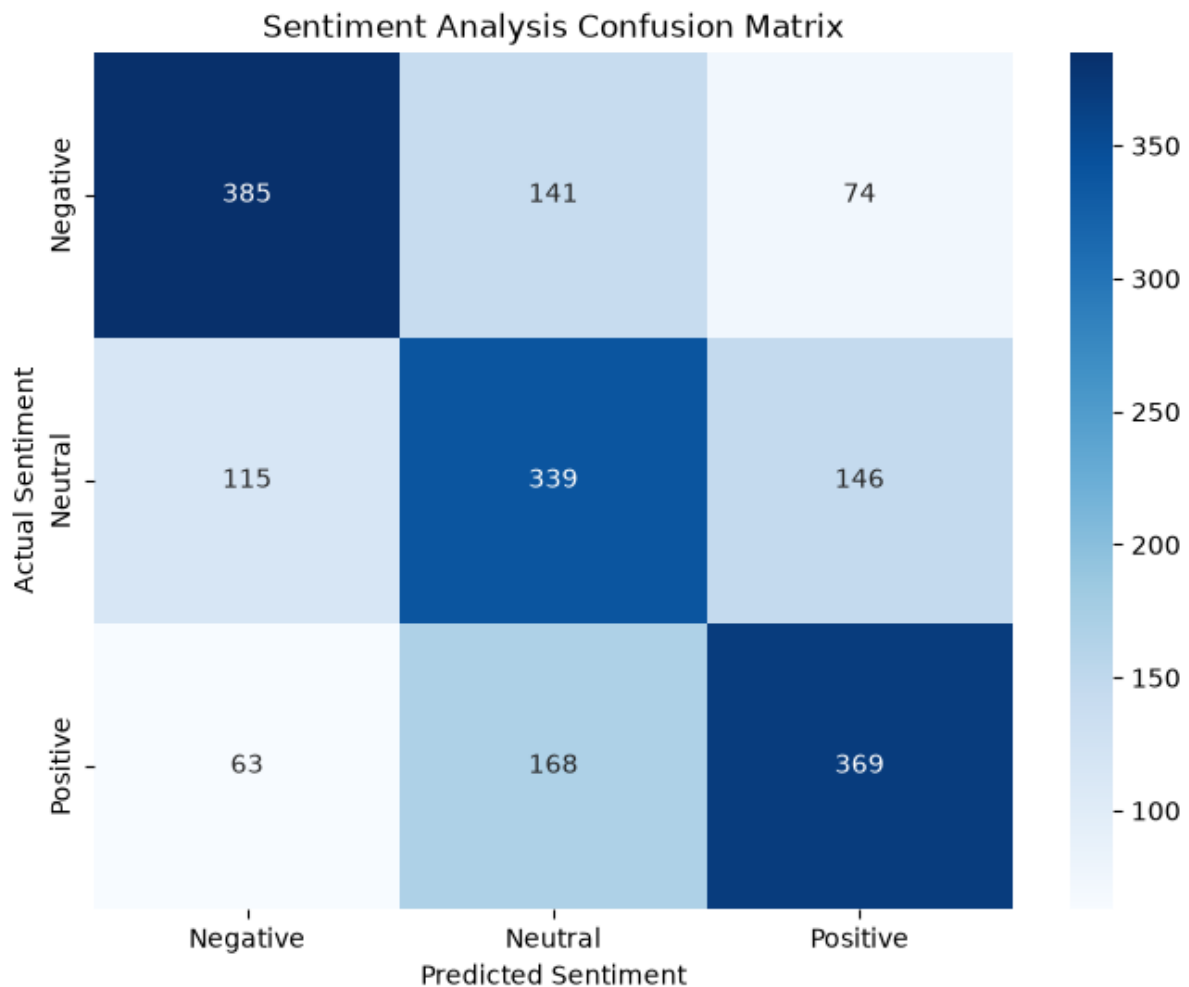
4. Training Methodology

The dataset of 9,000 labeled posts was split using an 80/20 ratio (7,200 training samples, 1,800 testing samples). We applied the `stratify=y` parameter during the split to ensure the exact class balance (33.3% for each sentiment) was preserved across both sets. The model was initialized with `class_weight='balanced'` and a maximum of 1,000 iterations to ensure optimal gradient convergence during the training phase.

5. Evaluation Metrics

The model achieved a robust baseline performance with an overall accuracy of **60.72%**. It performed best on Negative sentiments (Precision: 0.68, F1: 0.66) and struggled slightly more with Neutral sentiments (Precision: 0.52, F1: 0.54), highlighting the complexity of classifying text lacking strong emotional polarity.

6. Confusion Matrix



As illustrated in the generated confusion matrix, the model successfully identified 385 Negative, 339 Neutral, and 369 Positive posts. The highest misclassification overlap occurs between Neutral and Positive sentiments (e.g., 168 actual Positive posts predicted as Neutral, and 146 actual Neutral posts predicted as Positive). This indicates that polite or factual statements are often conflated with subtle positive sentiment.

7. Error Analysis

An analysis of the 707 misclassified instances out of 1,800 revealed specific contextual challenges for the TF-IDF bag-of-words approach:

- **Contextual Sarcasm & Emoticons:** In the post *"Damn my best friend is not going to riot Fest this Saturday since she has a test that night :(" (Actual: Negative, Predicted: Positive), the model heavily weighted the word "best" and failed to interpret the contextual sadness of the emoticon and the word "damn".*
- **Implicit Tone:** In *"John Cena throwing Jon Stewart at the WWE is pretty amusing"* (Actual: Positive, Predicted: Neutral), the model failed to capture the positive weight of the phrase "pretty amusing" within the specific context of entertainment.
- **Future Scope:** Transitioning from TF-IDF to advanced word embeddings (Word2Vec) or transformer architectures (BERT) would allow the Social Engine to understand sequential context, idioms, and sarcasm far more effectively.